Online-Retail-Data-Clustering -- Customer Segmentation

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [3]:
```
# setting to make numbers easier to read on display
pd.options.display.float_format = '{:,.2f}'.format

# show all columns on output
pd.set_option('display.max_columns', 999)
```

In [4]:
```
file_path = '/content/drive/MyDrive/Learning/DA-AB Testing/online_retail.xlsx'
```

Data Exploration

In [5]:
```
df = pd.read_excel(file_path, sheet_name=0)
```

In [6]:
```
df.head(10)
```

Out[6]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13,085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13,085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13,085.00	United Kingdom
5	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01 07:45:00	1.65	13,085.00	United Kingdom
6	489434	21871	SAVE THE PLANET MUG	24	2009-12-01 07:45:00	1.25	13,085.00	United Kingdom
7	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01 07:45:00	5.95	13,085.00	United Kingdom
8	489435	22350	CAT BOWL	12	2009-12-01 07:46:00	2.55	13,085.00	United Kingdom
9	489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-12-01 07:46:00	3.75	13,085.00	United Kingdom

Column description

- InvoiceNo: Invoice number. Nominal. A 6-digit integral number uniquely assigned to each transaction. If this code starts with the letter 'c', it indicates a cancellation.
- StockCode: Product (item) code. Nominal. A 5-digit integral number uniquely assigned to each distinct product.
- Description: Product (item) name. Nominal.
- Quantity: The quantities of each product (item) per transaction. Numeric.
- InvoiceDate: Invice date and time. Numeric. The day and time when a transaction was generated.
- UnitPrice: Unit price. Numeric. Product price per unit in sterling (Â£).
- CustomerID: Customer number. Nominal. A 5-digit integral number uniquely assigned to each customer.
- Country: Country name. Nominal. The name of the country where a customer resides.

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   Invoice      525461 non-null  object
 1   StockCode    525461 non-null  object
 2   Description  522533 non-null  object
 3   Quantity     525461 non-null  int64
 4   InvoiceDate  525461 non-null  datetime64[ns]
 5   Price        525461 non-null  float64
 6   Customer ID  417534 non-null  float64
 7   Country      525461 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 32.1+ MB
```

In [8]:
```
df.describe()
```

Out[8]:

	Quantity	InvoiceDate	Price	Customer ID
count	525,461.00	525461	525,461.00	417,534.00
mean	10.34	2010-06-28 11:37:36.845017856	4.69	15,360.65
min	-9,600.00	2009-12-01 07:45:00	-53,594.36	12,346.00
25%	1.00	2010-03-21 12:20:00	1.25	13,983.00
50%	3.00	2010-07-06 09:51:00	2.10	15,311.00
75%	10.00	2010-10-15 12:45:00	4.21	16,799.00
max	19,152.00	2010-12-09 20:01:00	25,111.09	18,287.00
std	107.42	NaN	146.13	1,680.81

In [9]:
```
df.describe(include='O')
```

Out[9]:

	Invoice	StockCode	Description	Country
count	525461	525461	522533	525461
unique	28816	4632	4681	40
top	537434	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
freq	675	3516	3549	485852

In [10]:
```
df[df['Customer ID'].isna()].head(10)
```

Out[10]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.00	NaN	United Kingdom
283	489463	71477	short	-240	2009-12-01 10:52:00	0.00	NaN	United Kingdom
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.00	NaN	United Kingdom
470	489521	21646	NaN	-50	2009-12-01 11:44:00	0.00	NaN	United Kingdom
577	489525	85226C	BLUE PULL BACK RACING CAR	1	2009-12-01 11:49:00	0.55	NaN	United Kingdom
578	489525	85227	SET/6 3D KIT CARDS FOR KIDS	1	2009-12-01 11:49:00	0.85	NaN	United Kingdom
1055	489548	22271	FELTCRAFT DOLL ROSIE	1	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1056	489548	22254	FELT TOADSTOOL LARGE	12	2009-12-01 12:32:00	1.25	NaN	United Kingdom
1057	489548	22273	FELTCRAFT DOLL MOLLY	3	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1058	489548	22195	LARGE HEART MEASURING SPOONS	1	2009-12-01 12:32:00	1.65	NaN	United Kingdom

In [11]:

```
df[df['Quantity']<0].head(10)
```

Out[11]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16,321.00	Australia
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16,321.00	Australia
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16,321.00	Australia
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16,321.00	Australia
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16,321.00	Australia
183	C489449	21871	SAVE THE PLANET MUG	-12	2009-12-01 10:33:00	1.25	16,321.00	Australia
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	2009-12-01 10:33:00	1.25	16,321.00	Australia
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	2009-12-01 10:33:00	0.85	16,321.00	Australia
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	2009-12-01 10:33:00	2.95	16,321.00	Australia
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	2009-12-01 10:44:00	4.25	17,592.00	United Kingdom

In [12]:

```
df['Invoice'] = df['Invoice'].astype(str)
df[df['Invoice'].str.match('^\\d{6}$') == False]
```

Out[12]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16,321.00	Australia
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16,321.00	Australia
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16,321.00	Australia
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16,321.00	Australia
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16,321.00	Australia
...	...	...	...	...	...	...	...	...
524695	C538123	22956	36 FOIL HEART CAKE CASES	-2	2010-12-09 15:41:00	2.10	12,605.00	Germany
524696	C538124	M	Manual	-4	2010-12-09 15:43:00	0.50	15,329.00	United Kingdom
524697	C538124	22699	ROSES REGENCY TEACUP AND SAUCER	-1	2010-12-09 15:43:00	2.95	15,329.00	United Kingdom
524698	C538124	22423	REGENCY CAKESTAND 3 TIER	-1	2010-12-09 15:43:00	12.75	15,329.00	United Kingdom
525282	C538164	35004B	SET OF 3 BLACK FLYING DUCKS	-1	2010-12-09 17:32:00	1.95	14,031.00	United Kingdom

10209 rows × 8 columns

In [13]:
```
df['Invoice'].str.replace("[0-9]","",regex = True).unique()
```

Out[13]:
```
array(['', 'C', 'A'], dtype=object)
```

In [14]:
```
df[df['Invoice'].str.startswith('A')]
```

Out[14]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
179403	A506401	B	Adjust bad debt	1	2010-04-29 13:36:00	-53,594.36	NaN	United Kingdom
276274	A516228	B	Adjust bad debt	1	2010-07-19 11:24:00	-44,031.79	NaN	United Kingdom
403472	A528059	B	Adjust bad debt	1	2010-10-20 12:04:00	-38,925.87	NaN	United Kingdom

In [15]:
```
df['StockCode'] = df['StockCode'].astype("str")
df[df['StockCode'].str.match("^\\d{5}$") == False]
```

Out[15]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
12	489436	48173C	DOOR MAT BLACK FLOCK	10	2009-12-01 09:06:00	5.95	13,078.00	United Kingdom
23	489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	2009-12-01 09:06:00	4.65	13,078.00	United Kingdom
28	489436	84596F	SMALL MARSHMALLOWS PINK BOWL	8	2009-12-01 09:06:00	1.25	13,078.00	United Kingdom
...	...	...	...	...	...	...	...	...
525387	538170	84029E	RED WOOLLY HOTTIE WHITE HEART.	2	2010-12-09 19:32:00	3.75	13,969.00	United Kingdom
525388	538170	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	2	2010-12-09 19:32:00	3.75	13,969.00	United Kingdom
525389	538170	85232B	SET OF 3 BABUSHKA STACKING TINS	2	2010-12-09 19:32:00	4.95	13,969.00	United Kingdom
525435	538171	47591D	PINK FAIRY CAKE CHILDRENS APRON	1	2010-12-09 20:01:00	1.95	17,530.00	United Kingdom
525436	538171	47591B	SCOTTIES CHILDRENS APRON	2	2010-12-09 20:01:00	1.65	17,530.00	United Kingdom

80112 rows × 8 columns

In [16]:

```
df[(df['StockCode'].str.match("^\\d{5}$") == False) & (df['StockCode'].str.match("^\\d{5}[a-z
```

Out[16]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
89	489439	POST	POSTAGE	3	2009-12-01 09:28:00	18.00	12,682.00	France
126	489444	POST	POSTAGE	1	2009-12-01 09:55:00	141.00	12,636.00	USA
173	489447	POST	POSTAGE	1	2009-12-01 10:10:00	130.00	12,362.00	Belgium
625	489526	POST	POSTAGE	6	2009-12-01 11:50:00	18.00	12,533.00	Germany
735	C489535	D	Discount	-1	2009-12-01 12:11:00	9.00	15,299.00	United Kingdom
...	...	...	...	...	...	...	...	...
524776	538147	M	Manual	1	2010-12-09 16:11:00	15.00	13,090.00	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

3098 rows × 8 columns

In [17]:
```
df[(df['StockCode'].str.match("^\\d{5}$") == False) & (df['StockCode'].str.match("^\\d{5}[a-
```

Out[17]:
```
array(['POST', 'D', 'DCGS0058', 'DCGS0068', 'DOT', 'M', 'DCGS0004',
       'DCGS0076', 'C2', 'BANK CHARGES', 'DCGS0003', 'TEST001',
       'gift_0001_80', 'DCGS0072', 'gift_0001_20', 'DCGS0044', 'TEST002',
       'gift_0001_10', 'gift_0001_50', 'DCGS0066N', 'gift_0001_30',
       'PADS', 'ADJUST', 'gift_0001_40', 'gift_0001_60', 'gift_0001_70',
       'gift_0001_90', 'DCGSSGIRL', 'DCGS0006', 'DCGS0016', 'DCGS0027',
       'DCGS0036', 'DCGS0039', 'DCGS0060', 'DCGS0056', 'DCGS0059', 'GIFT',
       'DCGSLBOY', 'm', 'DCGS0053', 'DCGS0062', 'DCGS0037', 'DCGSSBOY',
       'DCGSLGIRL', 'S', 'DCGS0069', 'DCGS0070', 'DCGS0075', 'B',
       'DCGS0041', 'ADJUST2', '47503J ', 'C3', 'SP1002', 'AMAZONFEE'],
      dtype=object)
```

In [18]:
```
df[df['StockCode'].str.contains('^DOT')]
```

Out[18]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2379	489597	DOT	DOTCOM POSTAGE	1	2009-12-01 14:28:00	647.19	NaN	United Kingdom
2539	489600	DOT	DOTCOM POSTAGE	1	2009-12-01 14:43:00	55.96	NaN	United Kingdom
2551	489601	DOT	DOTCOM POSTAGE	1	2009-12-01 14:44:00	68.39	NaN	United Kingdom
2571	489602	DOT	DOTCOM POSTAGE	1	2009-12-01 14:45:00	59.35	NaN	United Kingdom
2619	489603	DOT	DOTCOM POSTAGE	1	2009-12-01 14:46:00	42.39	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
524272	538071	DOT	DOTCOM POSTAGE	1	2010-12-09 14:09:00	885.94	NaN	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

736 rows × 8 columns

Code	Description	Action
DCGS	Negative quantities, missing customer ID	Exclude from clustering
D	Discount values	Exclude from clustering
DOT	Postage charges	Exclude from clustering
M / m	Manual transactions	Exclude from clustering
C2	Carriage transaction	Exclude from clustering
C3	Single unclear transaction	Exclude
BANK CHARGES	Bank charges	Exclude from clustering
B	Bank charges	Exclude from clustering
S	Samples sent to customer	Exclude from clustering
TESTXXX	Testing data	Exclude from clustering
gift__XXX	Gift card purchases, no customer data	Exclude
PADS	Valid stock code (padding)	Include
SP1002	Special request item, inconsistent pricing	Exclude for now
AMAZONFEE	Amazon-related fees	Exclude for now
ADJUSTX	Manual admin adjustments	Exclude for now

Data Cleaning

In [19]:
```
cleaned_df = df.copy()
```

Converts the Invoice column to string format and validates entries using a regex pattern. Retains only invoices containing exactly 6 digits, removing invalid or inconsistent records for cleaner analysis.

In [20]:
```
cleaned_df['Invoice'] = cleaned_df['Invoice'].astype('str')

mask = (
    cleaned_df['Invoice'].str.match("^\\d{6}$") == True
)

cleaned_df = cleaned_df[mask]
```

In [21]:
```
cleaned_df
```

Out[21]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13,085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13,085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13,085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17,530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17,530.00	United Kingdom

515252 rows × 8 columns

Converts StockCode to string and filters only valid product codes using regex patterns. Keeps codes with exactly 5 digits, 5 digits followed by letters, and the special valid code PADS. Removes non-product or invalid entries to ensure clean data for analysis.

In [22]:
```
cleaned_df['StockCode'] = cleaned_df['StockCode'].astype('str')

mask = (
    (cleaned_df['StockCode'].str.match("^\\d{5}$") == True)
    | (cleaned_df['StockCode'].str.match("^\\d{5}[a-zA-Z]+$") == True)
    | (cleaned_df['StockCode'].str.match("^PADS$") == True)
    )

cleaned_df = cleaned_df[mask]
```

In [23]:
```
cleaned_df
```

Out[23]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13,085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13,085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13,085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17,530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17,530.00	United Kingdom

512796 rows × 8 columns

In [24]:
```
cleaned_df.describe()
```

Out[24]:

	Quantity	InvoiceDate	Price	Customer ID
count	512,796.00	512796	512,796.00	406,337.00
mean	11.00	2010-06-28 18:26:52.709732352	3.39	15,373.63
min	-9,600.00	2009-12-01 07:45:00	0.00	12,346.00
25%	1.00	2010-03-21 13:27:00	1.25	14,004.00
50%	3.00	2010-07-06 14:25:00	2.10	15,326.00
75%	10.00	2010-10-15 14:50:00	4.21	16,814.00
max	19,152.00	2010-12-09 20:01:00	1,157.15	18,287.00
std	104.35	NaN	5.07	1,677.37

In [25]:
```
cleaned_df.dropna(subset=['Customer ID'], inplace=True)
```

In [26]:
```
cleaned_df.describe()
```

Out[26]:

	Quantity	InvoiceDate	Price	Customer ID
count	406,337.00	406337	406,337.00	406,337.00
mean	13.62	2010-07-01 10:11:06.543288320	2.99	15,373.63
min	1.00	2009-12-01 07:45:00	0.00	12,346.00
25%	2.00	2010-03-26 14:01:00	1.25	14,004.00
50%	5.00	2010-07-09 15:48:00	1.95	15,326.00
75%	12.00	2010-10-14 17:09:00	3.75	16,814.00
max	19,152.00	2010-12-09 20:01:00	295.00	18,287.00
std	97.00	NaN	4.29	1,677.37

In [27]:
```
cleaned_df[cleaned_df['Price'] == 0]
```

Out[27]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
4674	489825	22076	6 RIBBONS EMPIRE	12	2009-12-02 13:34:00	0.00	16,126.00	United Kingdom
6781	489998	48185	DOOR MAT FAIRY CAKE	2	2009-12-03 11:19:00	0.00	15,658.00	United Kingdom
18738	490961	22065	CHRISTMAS PUDDING TRINKET POT	1	2009-12-08 15:25:00	0.00	14,108.00	United Kingdom
18739	490961	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-12-08 15:25:00	0.00	14,108.00	United Kingdom
32916	492079	85042	ANTIQUE LILY FAIRY LIGHTS	8	2009-12-15 13:49:00	0.00	15,070.00	United Kingdom
40101	492760	21143	ANTIQUE GLASS HEART DECORATION	12	2009-12-18 14:22:00	0.00	18,071.00	United Kingdom
47126	493761	79320	FLAMINGO LIGHTS	24	2010-01-06 14:54:00	0.00	14,258.00	United Kingdom
48342	493899	22355	CHARLOTTE BAG , SUKI DESIGN	10	2010-01-08 10:43:00	0.00	12,417.00	Belgium
57619	494607	21533	RETRO SPOT LARGE MILK JUG	12	2010-01-15 12:43:00	0.00	16,858.00	United Kingdom
111348	500073	21662	VINTAGE GLASS COFFEE CADDY	1	2010-03-04 11:44:00	0.00	13,047.00	United Kingdom
149201	503585	22459	CAST IRON HOOK GARDEN TROWEL	8	2010-04-01 17:13:00	0.00	13,047.00	United Kingdom
149202	503585	22458	CAST IRON HOOK GARDEN FORK	8	2010-04-01 17:13:00	0.00	13,047.00	United Kingdom
166143	505083	22376	AIRLINE BAG VINTAGE JET SET WHITE	1	2010-04-20 09:56:00	0.00	12,623.00	Germany
232526	511902	21765	HANGING METAL BIRD BATH	1	2010-06-11 11:12:00	0.00	12,748.00	United Kingdom
240455	512609	20914	SET/5 RED SPOTTY LID GLASS BOWLS	2	2010-06-17 10:12:00	0.00	14,045.00	United Kingdom
248583	513416	22423	REGENCY CAKESTAND 3 TIER	5	2010-06-24 12:34:00	0.00	13,089.00	United Kingdom
276858	516304	22690	DOORMAT HOME SWEET	6	2010-07-19 13:13:00	0.00	14,025.00	United Kingdom

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
			HOME BLUE					
296375	518231	22472	TV DINNER TRAY DOLLY GIRL	9	2010-08-05 15:28:00	0.00	12,471.00	Germany
327801	521375	22202	MILK PAN PINK RETROSPOT	3	2010-09-05 11:58:00	0.00	12,647.00	Germany
358820	524181	46000M	POLYESTER FILLER PAD 45x45cm	648	2010-09-27 16:59:00	0.00	17,450.00	United Kingdom
364333	524701	22218	CAKE STAND LACE WHITE	2	2010-09-30 12:19:00	0.00	17,667.00	United Kingdom
392008	527084	22630	DOLLY GIRL LUNCH BOX	64	2010-10-14 15:33:00	0.00	14,646.00	Netherlands
400047	527696	22121	NOEL WOODEN BLOCK LETTERS	1	2010-10-18 15:13:00	0.00	13,554.00	United Kingdom
439309	531361	21843	RED RETROSPOT CAKE STAND	2	2010-11-07 14:26:00	0.00	12,820.00	United Kingdom
453705	532470	22624	IVORY KITCHEN SCALES	2	2010-11-12 11:41:00	0.00	12,647.00	Germany
471775	533822	22846	BREAD BIN DINER STYLE RED	1	2010-11-19 09:40:00	0.00	12,647.00	Germany
471776	533822	22845	VINTAGE CREAM CAT FOOD CONTAINER	1	2010-11-19 09:40:00	0.00	12,647.00	Germany
512240	537197	22841	ROUND CAKE TIN VINTAGE GREEN	1	2010-12-05 14:02:00	0.00	12,647.00	Germany

In [28]:
```
len(cleaned_df[cleaned_df['Price'] == 0])
```

Out[28]: 28

In [29]:
```
cleaned_df = cleaned_df[cleaned_df['Price'] > 0]
```

In [30]:
```
cleaned_df.describe()
```

Out[30]:

	Quantity	InvoiceDate	Price	Customer ID
count	406,309.00	406309	406,309.00	406,309.00
mean	13.62	2010-07-01 10:14:25.869572352	2.99	15,373.72
min	1.00	2009-12-01 07:45:00	0.00	12,346.00
25%	2.00	2010-03-26 14:01:00	1.25	14,006.00
50%	5.00	2010-07-09 15:48:00	1.95	15,326.00
75%	12.00	2010-10-14 17:09:00	3.75	16,814.00
max	19,152.00	2010-12-09 20:01:00	295.00	18,287.00
std	97.00	NaN	4.29	1,677.33

In [31]:
```
cleaned_df['Price'].min()
```

Out[31]:
```
0.001
```

In [32]:
```
len(cleaned_df)/len(df)
```

Out[32]:
```
0.7732429238325965
```

Dropped about 23% of records during cleaning

Feature Engineering

In [34]:
```
cleaned_df['SalesLineTotal'] = cleaned_df['Quantity']*cleaned_df['Price']
cleaned_df
```

Out[34]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	SalesLin
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13,085.00	United Kingdom	
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom	
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13,085.00	United Kingdom	
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13,085.00	United Kingdom	
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13,085.00	United Kingdom	
...	...	...	...	...	...	...	...	...	
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17,530.00	United Kingdom	
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom	
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom	
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17,530.00	United Kingdom	
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17,530.00	United Kingdom	

406309 rows × 9 columns

In [35]:

```
aggregated_df = cleaned_df.groupby(by='Customer ID', as_index=False).agg(
    Monetoryvalue=('SalesLineTotal', 'sum'),
    Frequency=('Invoice', 'nunique'),
    LastInvoicedate=('InvoiceDate','max')
)

aggregated_df.head(5)
```

Out[35]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate
0	12,346.00	169.36	2	2010-06-28 13:53:00
1	12,347.00	1,323.32	2	2010-12-07 14:57:00
2	12,348.00	221.16	1	2010-09-27 14:59:00
3	12,349.00	2,221.14	2	2010-10-28 08:23:00
4	12,351.00	300.93	1	2010-11-29 15:23:00

In [36]:

```
max_invoice_date = aggregated_df['LastInvoicedate'].max()
max_invoice_date
```

Out[36]:

```
Timestamp('2010-12-09 20:01:00')
```

In [38]:

```
aggregated_df['Recency'] = (max_invoice_date - aggregated_df['LastInvoicedate']).dt.days
aggregated_df
```

Out[38]:

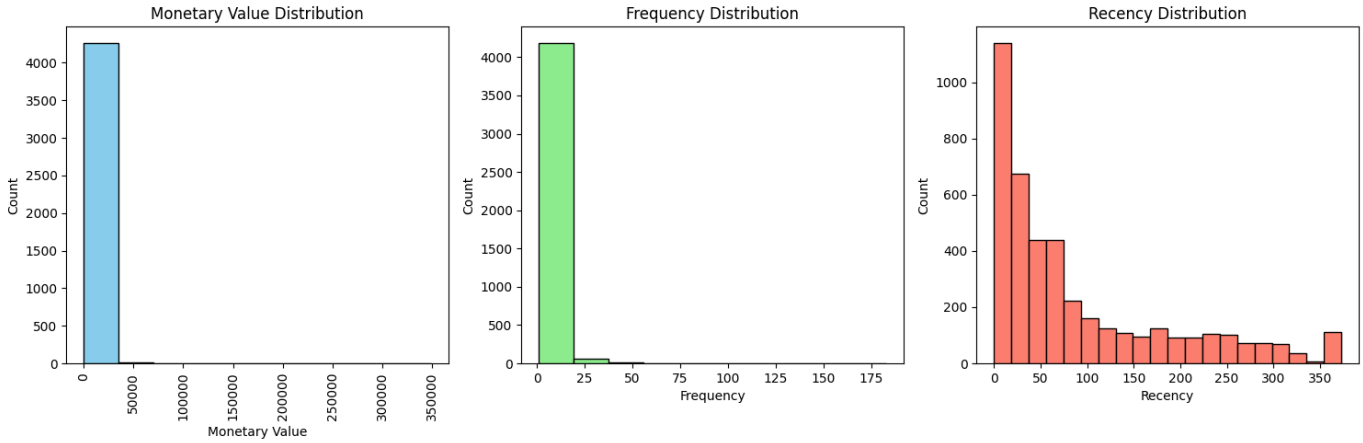
	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
0	12,346.00	169.36	2	2010-06-28 13:53:00	164
1	12,347.00	1,323.32	2	2010-12-07 14:57:00	2
2	12,348.00	221.16	1	2010-09-27 14:59:00	73
3	12,349.00	2,221.14	2	2010-10-28 08:23:00	42
4	12,351.00	300.93	1	2010-11-29 15:23:00	10
...	...	...	...	...	...
4280	18,283.00	641.77	6	2010-11-22 15:30:00	17
4281	18,284.00	411.68	1	2010-10-04 11:33:00	66
4282	18,285.00	377.00	1	2010-02-17 10:24:00	295
4283	18,286.00	1,246.43	2	2010-08-20 11:57:00	111
4284	18,287.00	2,295.71	4	2010-11-22 11:51:00	17

4285 rows × 5 columns

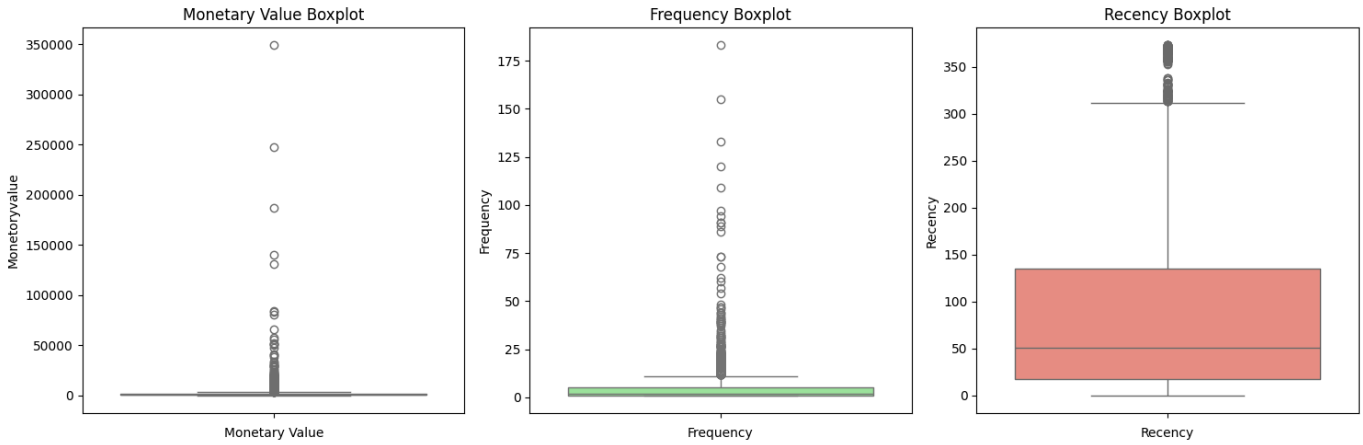
In [42]:

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.hist(aggregated_df['Monetoryvalue'], bins=10, color='skyblue', edgecolor='black')
plt.title('Monetary Value Distribution')
plt.xlabel('Monetary Value')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.tight_layout()

plt.subplot(1, 3, 2)
plt.hist(aggregated_df['Frequency'], bins=10, color='lightgreen', edgecolor='black')
plt.title('Frequency Distribution')
plt.xlabel('Frequency')
plt.ylabel('Count')

plt.subplot(1, 3, 3)
plt.hist(aggregated_df['Recency'], bins=20, color='salmon', edgecolor='black')
plt.title('Recency Distribution')
```

```
plt.xlabel('Recency')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



In [44]:
```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=aggregated_df['Monetoryvalue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=aggregated_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=aggregated_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```



In [47]:
```
M_Q1 = aggregated_df['Monetoryvalue'].quantile(0.25)
M_Q3 = aggregated_df['Monetoryvalue'].quantile(0.75)

M_IQR = M_Q3 - M_Q1

M_Lower_Bound = M_Q1 - 1.5 * M_IQR
M_Upper_Bound = M_Q3 + 1.5 * M_IQR
```

In [48]:
```
monetary_outliers_df = aggregated_df[
    (aggregated_df['Monetoryvalue'] < M_Lower_Bound) |
    (aggregated_df['Monetoryvalue'] > M_Upper_Bound)
```

```
]

monetary_outliers_df
```

Out[48]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
9	12,357.00	11,229.99	1	2010-11-16 10:05:00	23
25	12,380.00	4,782.84	4	2010-08-31 14:54:00	100
42	12,409.00	12,346.62	4	2010-10-15 10:24:00	55
48	12,415.00	19,468.84	4	2010-11-29 15:07:00	10
61	12,431.00	4,145.52	11	2010-12-01 10:03:00	8
...	...	...	...	...	...
4236	18,225.00	7,545.14	15	2010-12-09 15:46:00	0
4237	18,226.00	6,650.83	15	2010-11-26 15:51:00	13
4241	18,231.00	4,791.80	23	2010-10-29 14:17:00	41
4253	18,251.00	21,964.14	8	2010-09-20 13:26:00	80
4262	18,260.00	7,318.91	17	2010-11-30 12:25:00	9

423 rows × 5 columns

In [49]:

```
monetary_outliers_df.describe()
```

Out[49]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
count	423.00	423.00	423.00	423	423.00
mean	15,103.04	12,188.10	17.17	2010-11-09 12:26:02.978723328	30.04
min	12,357.00	3,802.04	1.00	2009-12-10 18:03:00	0.00
25%	13,622.00	4,605.94	8.00	2010-11-08 13:17:30	3.00
50%	14,961.00	6,191.32	12.00	2010-11-26 12:19:00	13.00
75%	16,692.00	10,273.24	18.00	2010-12-06 10:34:30	31.00
max	18,260.00	349,164.35	183.00	2010-12-09 19:32:00	364.00
std	1,728.66	25,830.85	19.73	NaN	51.54

In [51]:

```
F_Q1 = aggregated_df['Frequency'].quantile(0.25)
F_Q3 = aggregated_df['Frequency'].quantile(0.75)

F_IQR = F_Q3 - F_Q1

F_lower_bound = F_Q1 - 1.5*F_IQR
F_upper_bound = F_Q3 + 1.5*F_IQR
```

In [52]:

```
frequenct_outlier_df = aggregated_df[
    (aggregated_df['Frequency'] < F_lower_bound) |
    (aggregated_df['Frequency'] > F_upper_bound)
]

frequenct_outlier_df
```

Out[52]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
65	12,437.00	6,834.99	20	2010-11-09 14:46:00	30
84	12,471.00	17,721.45	44	2010-11-30 14:35:00	9
85	12,472.00	10,426.48	13	2010-12-05 14:19:00	4
92	12,482.00	21,941.72	27	2010-05-12 16:51:00	211
115	12,523.00	2,330.38	12	2010-11-30 12:31:00	9
...	...	...	...	...	...
4236	18,225.00	7,545.14	15	2010-12-09 15:46:00	0
4237	18,226.00	6,650.83	15	2010-11-26 15:51:00	13
4241	18,231.00	4,791.80	23	2010-10-29 14:17:00	41
4250	18,245.00	3,757.92	13	2010-11-25 16:52:00	14
4262	18,260.00	7,318.91	17	2010-11-30 12:25:00	9

279 rows × 5 columns

In [54]:

```
frequenct_outlier_df.describe()
```

Out[54]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
count	279.00	279.00	279.00	279	279.00
mean	15,352.66	14,409.71	23.81	2010-11-23 11:06:20.645161216	16.09
min	12,437.00	1,094.39	12.00	2010-05-12 16:51:00	0.00
25%	13,800.00	4,331.56	13.00	2010-11-20 13:14:30	2.00
50%	15,465.00	6,615.77	17.00	2010-12-02 10:46:00	7.00
75%	16,828.50	11,692.41	23.00	2010-12-07 11:08:30	19.00
max	18,260.00	349,164.35	183.00	2010-12-09 19:32:00	211.00
std	1,748.43	31,381.74	21.93	NaN	26.59

In [56]:

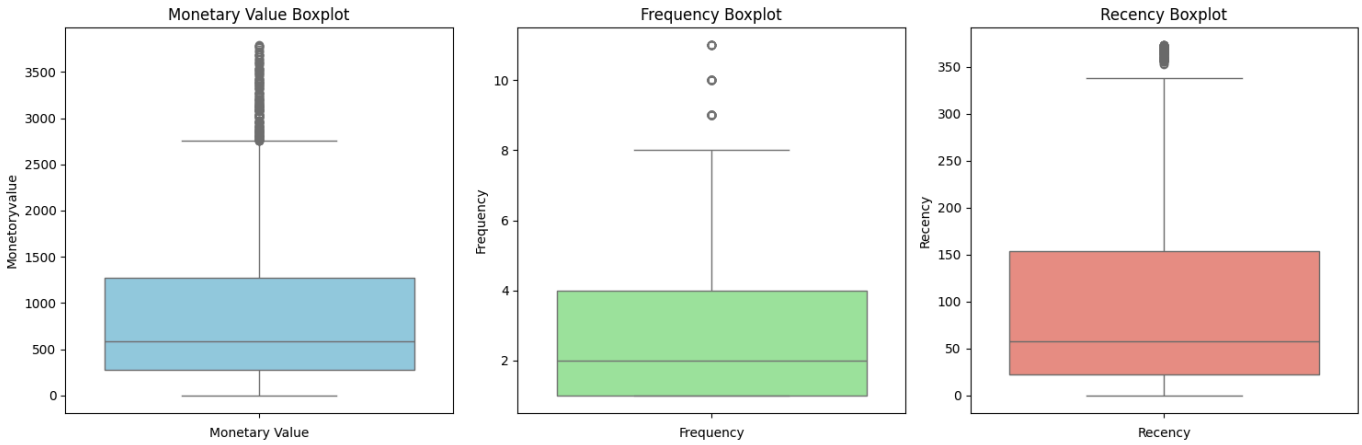
```
non_outliers_df = aggregated_df[(~aggregated_df.index.isin(monetary_outliers_df.index)) & (~a

non_outliers_df.describe()
```

Out[56]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency
count	3,809.00	3,809.00	3,809.00	3809	3,809.00
mean	15,376.48	885.50	2.86	2010-09-03 11:16:46.516146176	97.08
min	12,346.00	1.55	1.00	2009-12-01 10:49:00	0.00
25%	13,912.00	279.91	1.00	2010-07-08 14:48:00	22.00
50%	15,389.00	588.05	2.00	2010-10-12 16:25:00	58.00
75%	16,854.00	1,269.05	4.00	2010-11-17 13:14:00	154.00
max	18,287.00	3,788.21	11.00	2010-12-09 20:01:00	373.00
std	1,693.20	817.67	2.24	NaN	98.11

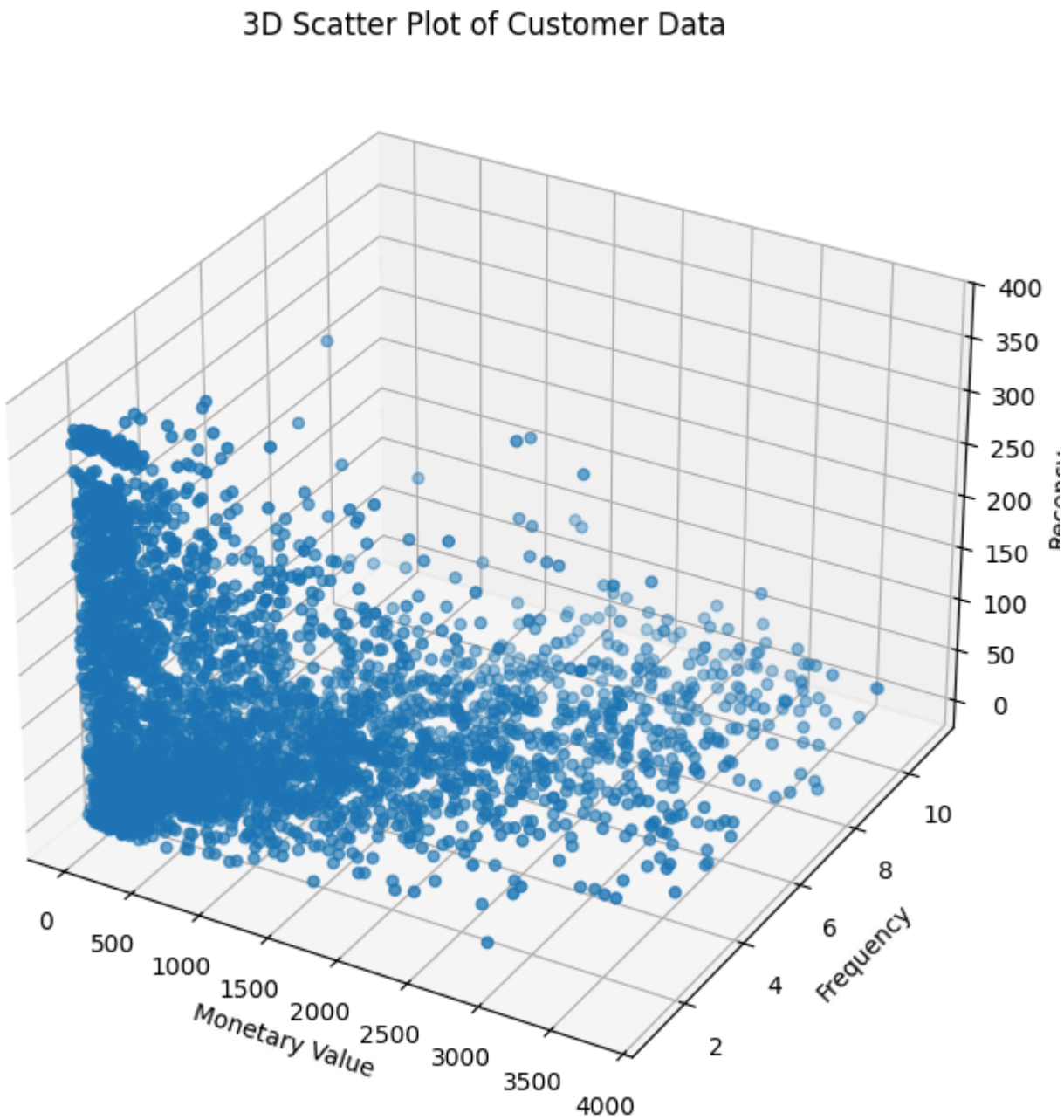
In [60]:

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=non_outliers_df['Monetoryvalue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=non_outliers_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=non_outliers_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```



In [62]:

```
fig = plt.figure(figsize=(8, 8))

ax = fig.add_subplot(projection="3d")

scatter = ax.scatter(non_outliers_df["Monetoryvalue"], non_outliers_df["Frequency"], non_outl

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()
```



📌 Standard Scaling

Standard scaling transforms the features of your data so that they have:

- **Mean = 0**
- **Standard Deviation = 1**

This ensures that each feature contributes equally to the analysis.

📐 Formula

$$z = \frac{x - \mu}{\sigma}$$

🔍 Where:

- **z** = standardized value
- **x** = original value
- **μ (mu)** = mean of the feature
- **σ (sigma)** = standard deviation of the feature

In [64]:
```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(non_outliers_df[["Monetoryvalue", "Frequency", "Recency"]]

scaled_data
```

Out[64]:
```
array([[-0.87594534, -0.38488934,  0.68214853],
       [ 0.5355144 , -0.38488934, -0.96925093],
       [-0.81258645, -0.83063076, -0.24548944],
       ...,
       [-0.62197163, -0.83063076,  2.01753946],
       [ 0.44146683, -0.38488934,  0.14187587],
       [ 1.72488781,  0.50659348, -0.81634357]])
```

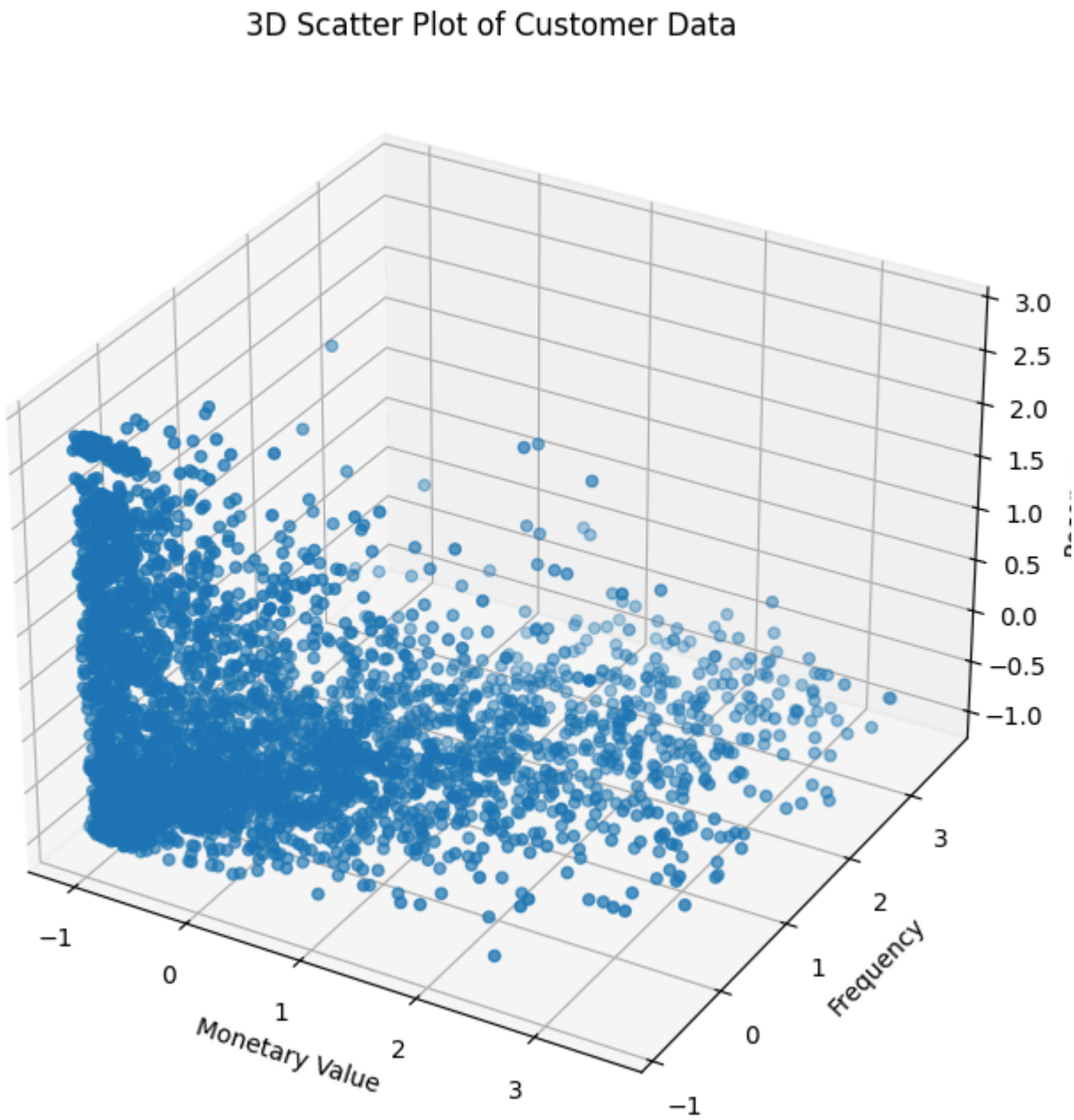
In [65]:
```
scaled_data_df = pd.DataFrame(scaled_data, index=non_outliers_df.index, columns=("MonetaryVal

scaled_data_df
```

Out[65]:

	MonetaryValue	Frequency	Recency
0	-0.88	-0.38	0.68
1	0.54	-0.38	-0.97
2	-0.81	-0.83	-0.25
3	1.63	-0.38	-0.56
4	-0.72	-0.83	-0.89
...	...	...	...
4280	-0.30	1.40	-0.82
4281	-0.58	-0.83	-0.32
4282	-0.62	-0.83	2.02
4283	0.44	-0.38	0.14
4284	1.72	0.51	-0.82

3809 rows × 3 columns

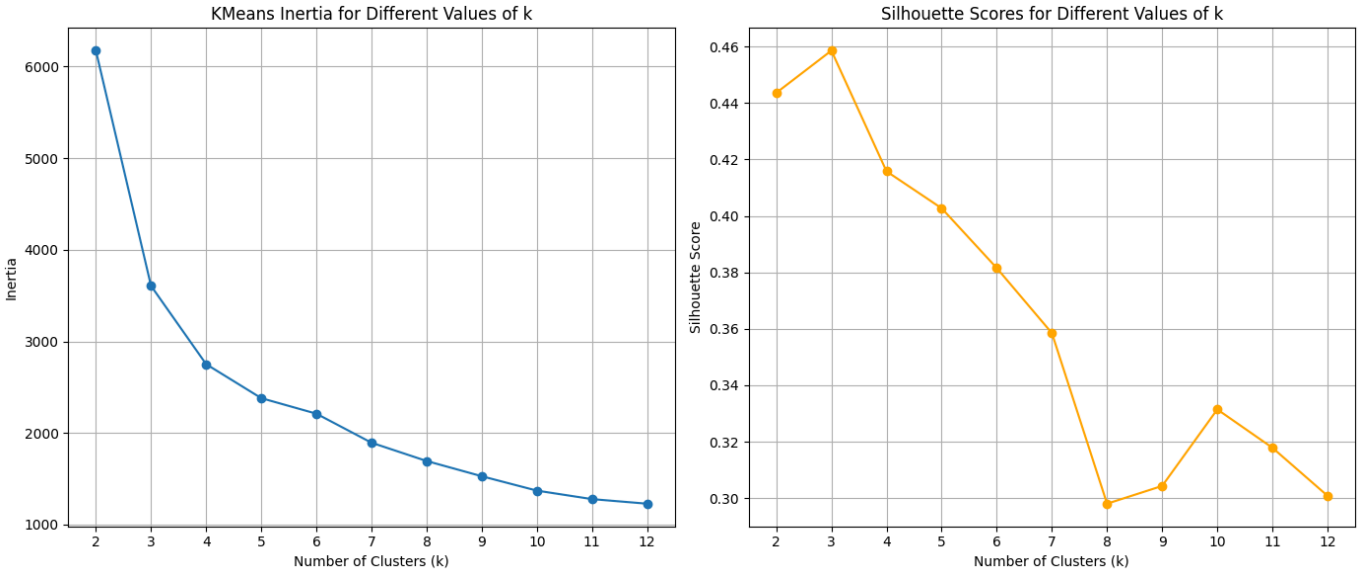
In [66]:
```
fig = plt.figure(figsize=(8, 8))

ax = fig.add_subplot(projection="3d")

scatter = ax.scatter(scaled_data_df["MonetaryValue"], scaled_data_df["Frequency"], scaled_dat

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()
```



KMeans Clustering

In [67]:
```
max_k = 12

inertia = []
silhoutte_scores = []
k_values = range(2, max_k + 1)

for k in k_values:

    kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)

    cluster_labels = kmeans.fit_predict(scaled_data_df)

    sil_score = silhouette_score(scaled_data_df, cluster_labels)

    silhoutte_scores.append(sil_score)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(k_values, inertia, marker='o')
```

```
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(k_values, silhoutte_scores, marker='o', color='orange')
plt.title('Silhouette Scores for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.grid(True)

plt.tight_layout()
plt.show()
```



📊 Silhouette Score

The Silhouette Score measures how well a data point fits within its cluster compared to other clusters.

📐 Formula

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

🔍 Where:

- **s(i)** = silhouette score for a single sample i
- **a(i)** = average distance between i and all other points in the same cluster
- **b(i)** = minimum average distance between i and all points in the nearest cluster to which i does not belong

🎯 Range:

- The score ranges from **-1 to 1**
  - **+1** → well-clustered
  - **0** → overlapping clusters
  - **-1** → misclassified points

Higher values indicate more distinct and well-separated clusters.

In [68]:
```
kmeans = KMeans(n_clusters=4, random_state=42, max_iter=1000)

cluster_labels = kmeans.fit_predict(scaled_data_df)

cluster_labels
```

Out[68]:
```
array([1, 0, 2, ..., 1, 0, 0], dtype=int32)
```

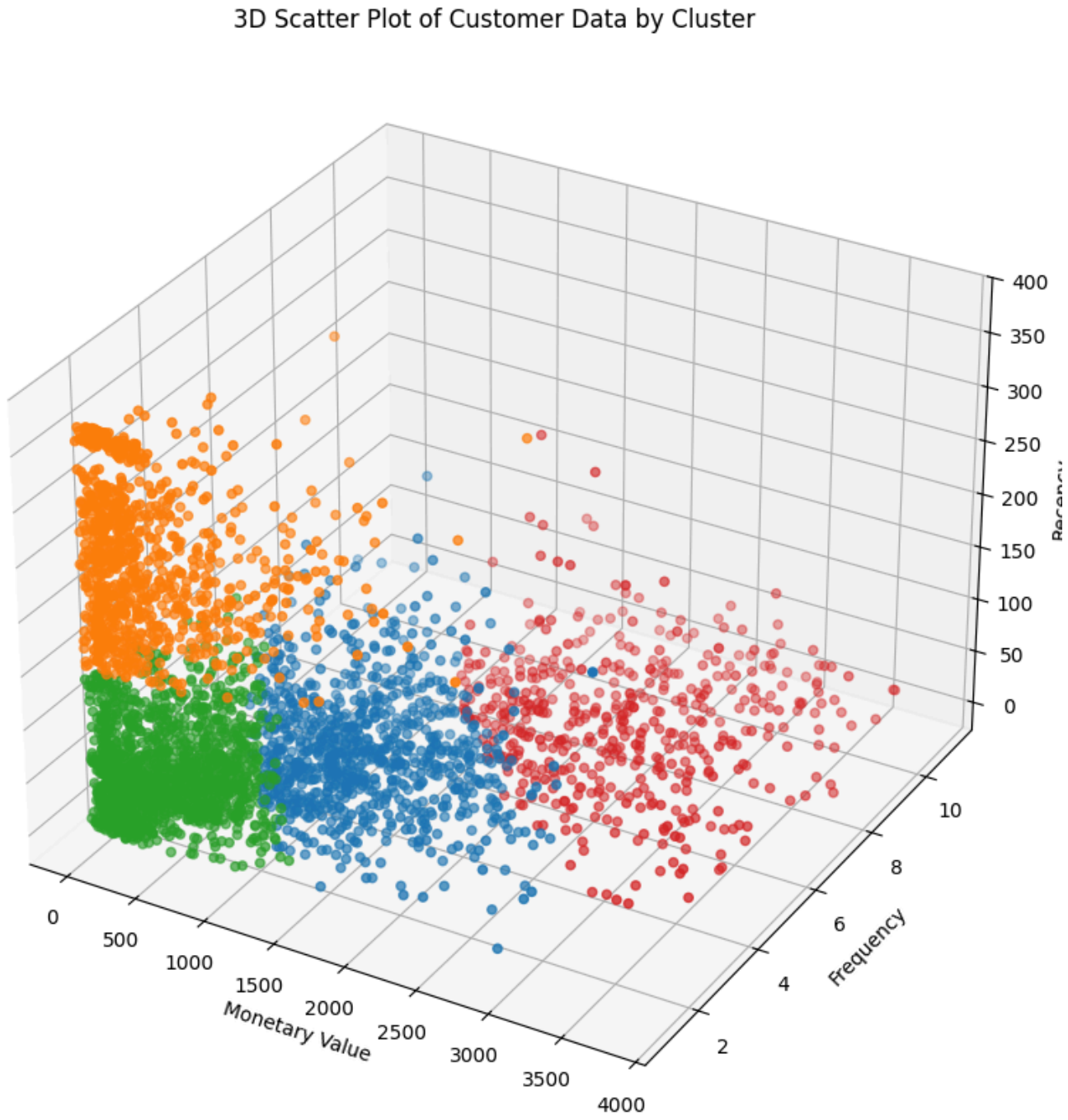
In [69]:
```
non_outliers_df["Cluster"] = cluster_labels

non_outliers_df
```

Out[69]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency	Cluster
0	12,346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12,347.00	1,323.32	2	2010-12-07 14:57:00	2	0
2	12,348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12,349.00	2,221.14	2	2010-10-28 08:23:00	42	0
4	12,351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4280	18,283.00	641.77	6	2010-11-22 15:30:00	17	0
4281	18,284.00	411.68	1	2010-10-04 11:33:00	66	2
4282	18,285.00	377.00	1	2010-02-17 10:24:00	295	1
4283	18,286.00	1,246.43	2	2010-08-20 11:57:00	111	0
4284	18,287.00	2,295.71	4	2010-11-22 11:51:00	17	0

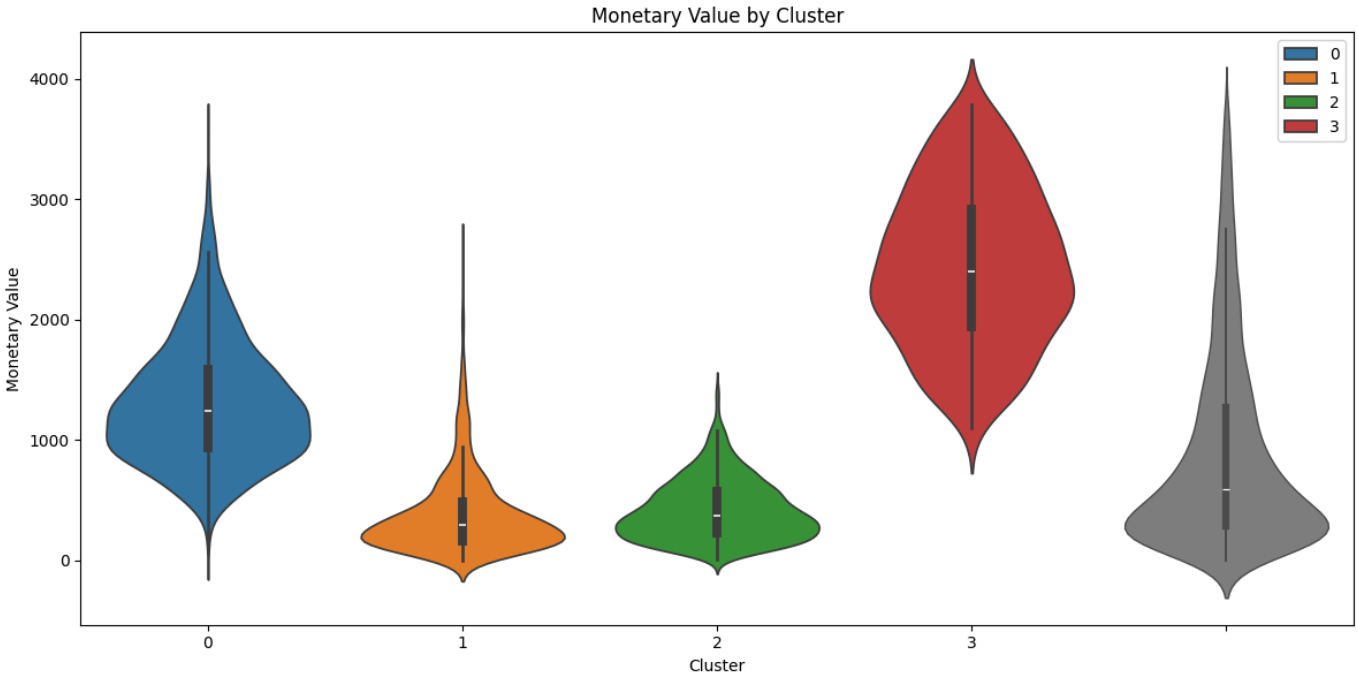
3809 rows × 6 columns

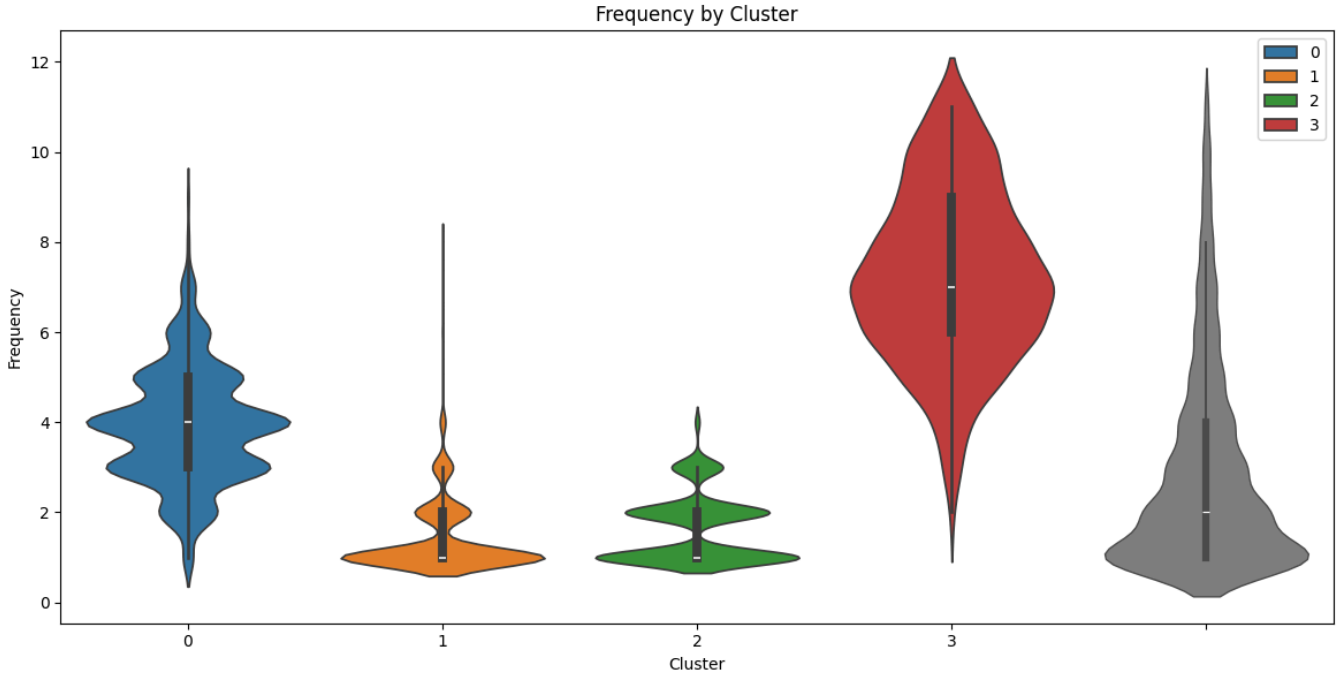
In [71]:
```
cluster_colors = {0: '#1f77b4',  # Blue
                  1: '#ff7f0e',  # Orange
                  2: '#2ca02c',  # Green
                  3: '#d62728'}  # Red

colors = non_outliers_df['Cluster'].map(cluster_colors)

fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(non_outliers_df['Monetoryvalue'],
                     non_outliers_df['Frequency'],
                     non_outliers_df['Recency'],
                     c=colors,  # Use mapped solid colors
                     marker='o')

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data by Cluster')

plt.show()
```

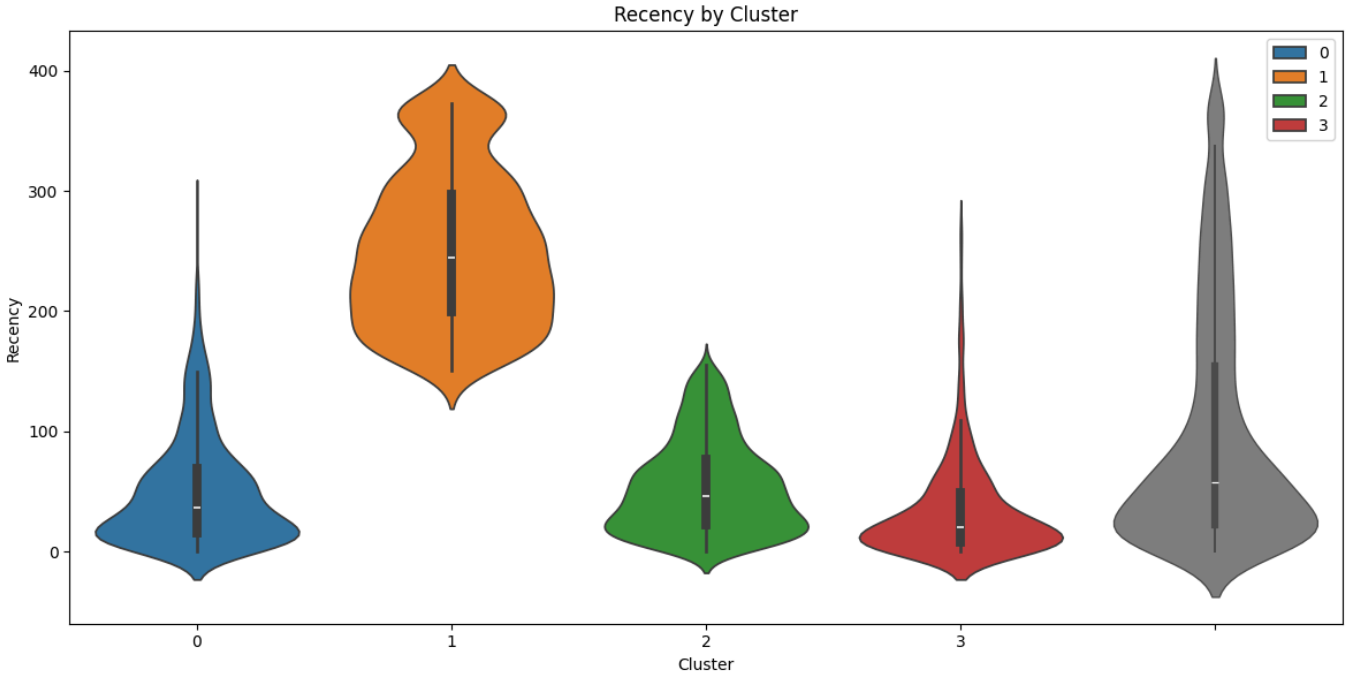


In [72]:

```
plt.figure(figsize=(12, 18))

plt.subplot(3, 1, 1)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Monetoryvalue'], palette=clus
sns.violinplot(y=non_outliers_df['Monetoryvalue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Frequency'], palette=cluster_
sns.violinplot(y=non_outliers_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')


plt.subplot(3, 1, 3)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Recency'], palette=cluster_co
sns.violinplot(y=non_outliers_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```


Monetary Value by Cluster
Monetary Value
Cluster
0
1
2
3


Frequency by Cluster
Frequency
Cluster
0
1
2
3


Recency by Cluster
Recency
Cluster
0
1
2
3

📊 Customer Segmentation (Cluster Interpretation)

🔵 Cluster 0: **Retain**

Rationale:
This cluster represents high-value customers who purchase regularly, though not always very recently. The focus should be on retention efforts to maintain their loyalty and spending levels.

Action:
Implement loyalty programs, personalized offers, and regular engagement to ensure they remain active.

🟠 Cluster 1: **Re-Engage**

Rationale:
This group includes lower-value, infrequent buyers who haven't purchased recently. The focus should be on re-engagement to bring them back into active purchasing behavior.

Action:
Use targeted marketing campaigns, special discounts, or reminders to encourage them to return and purchase again.

🟢 Cluster 2: **Nurture**

Rationale:
This cluster represents the least active and lowest-value customers, but they have made recent purchases. These customers may be new or need nurturing to increase their engagement and spending.

Action:
Focus on building relationships, providing excellent customer service, and offering incentives to encourage more frequent purchases.

🔴 Cluster 3: **Reward**

Rationale:
This cluster includes high-value, very frequent buyers, many of whom are still actively purchasing. They are your most loyal customers, and rewarding their loyalty is key to maintaining their engagement.

Action:
Implement a robust loyalty program, provide exclusive offers, and recognize their loyalty to keep them engaged and satisfied.

📌 Summary of Cluster Names

Cluster	Color	Segment Name
0	Blue	Retain
1	Orange	Re-Engage

Cluster	Color	Segment Name
2	Green	Nurture
3	Red	Reward

In [77]:
```
overlap_indices = monetary_outliers_df.index.intersection(frequency_outliers_df.index)

monetary_only_outliers = monetary_outliers_df.drop(overlap_indices)
frequency_only_outliers = frequency_outliers_df.drop(overlap_indices)
monetary_and_frequency_outliers = monetary_outliers_df.loc[overlap_indices]

monetary_only_outliers["Cluster"] = -1
frequency_only_outliers["Cluster"] = -2
monetary_and_frequency_outliers["Cluster"] = -3

outlier_clusters_df = pd.concat([monetary_only_outliers, frequency_only_outliers, monetary_an

outlier_clusters_df
```

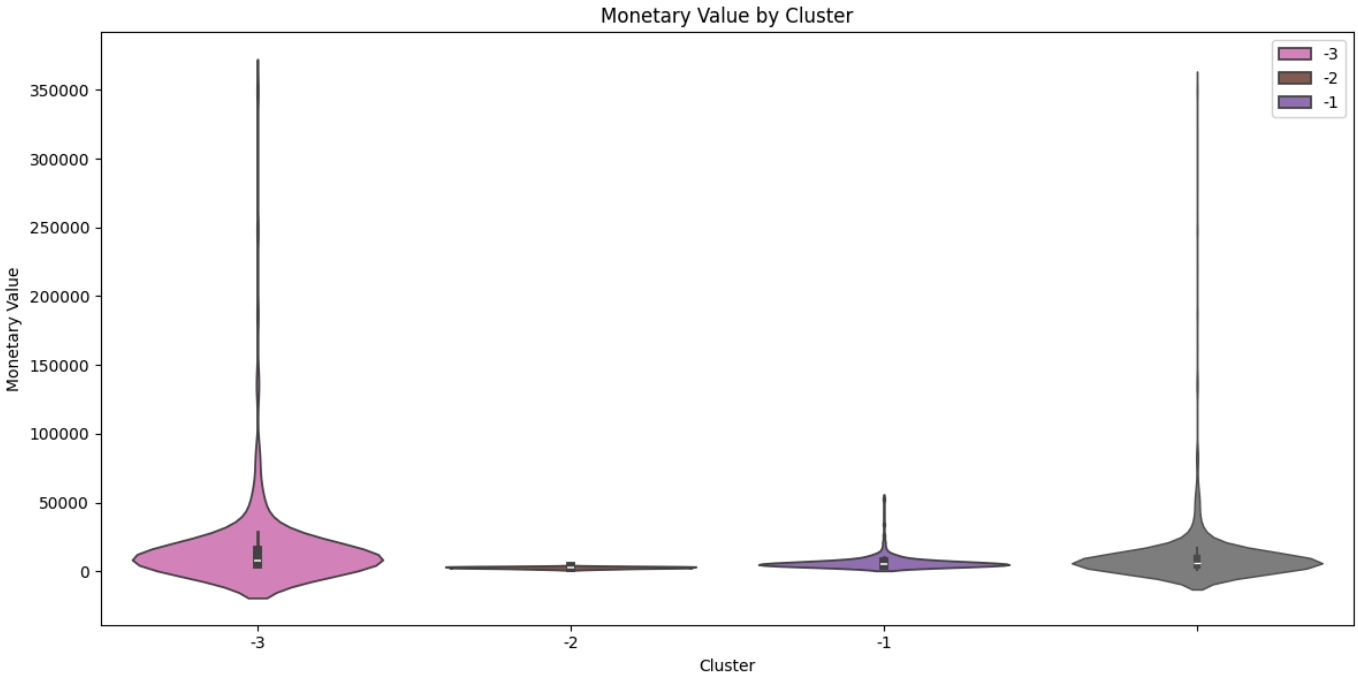
Out[77]:

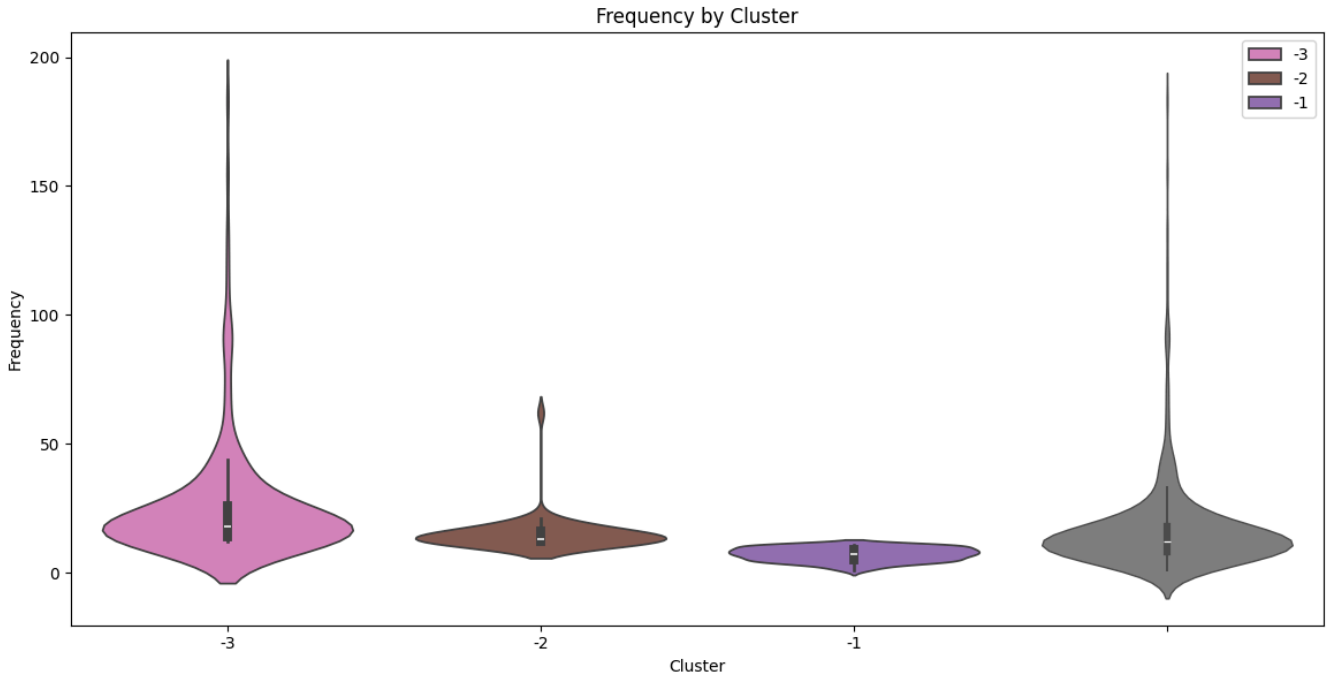
	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency	Cluster
9	12,357.00	11,229.99	1	2010-11-16 10:05:00	23	-1
25	12,380.00	4,782.84	4	2010-08-31 14:54:00	100	-1
42	12,409.00	12,346.62	4	2010-10-15 10:24:00	55	-1
48	12,415.00	19,468.84	4	2010-11-29 15:07:00	10	-1
61	12,431.00	4,145.52	11	2010-12-01 10:03:00	8	-1
...	...	...	...	...	...	...
4235	18,223.00	7,516.31	12	2010-11-17 12:20:00	22	-3
4236	18,225.00	7,545.14	15	2010-12-09 15:46:00	0	-3
4237	18,226.00	6,650.83	15	2010-11-26 15:51:00	13	-3
4241	18,231.00	4,791.80	23	2010-10-29 14:17:00	41	-3
4262	18,260.00	7,318.91	17	2010-11-30 12:25:00	9	-3

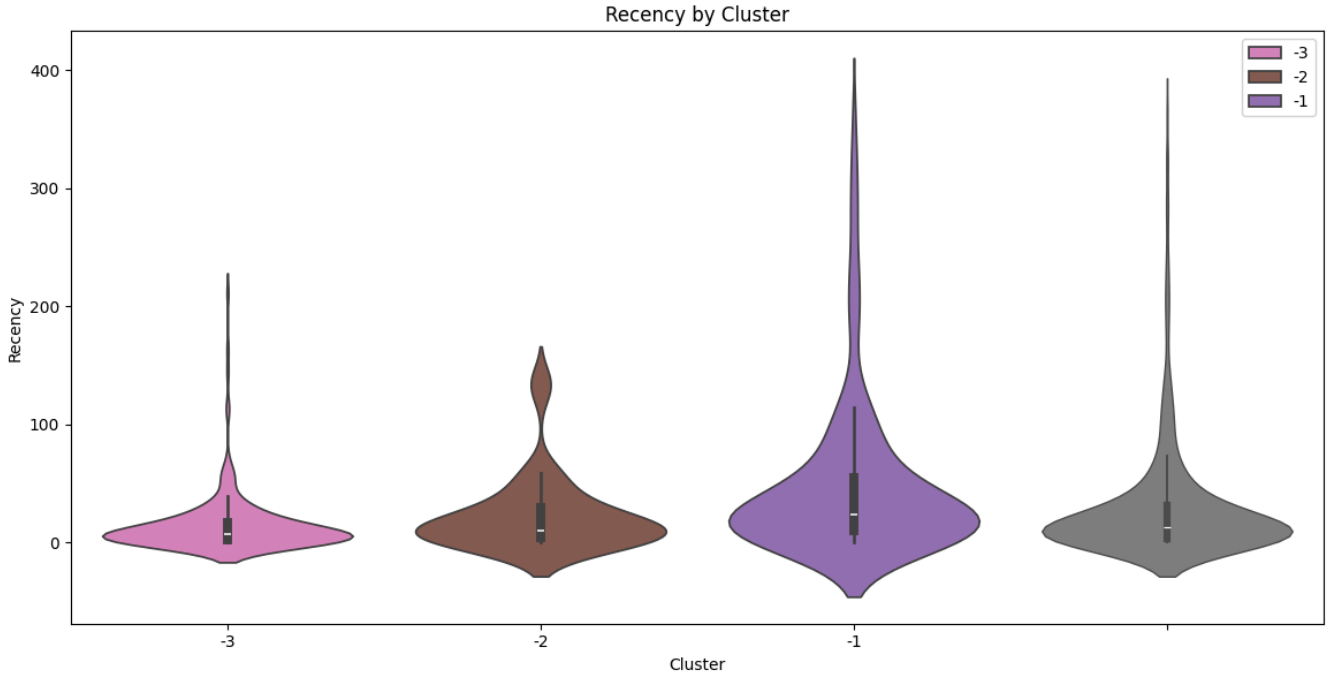
476 rows × 6 columns

In [79]:
```
cluster_colors = {-1: '#9467bd',
                  -2: '#8c564b',
                  -3: '#e377c2'}

plt.figure(figsize=(12, 18))

plt.subplot(3, 1, 1)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Monetoryvalue'], pale
sns.violinplot(y=outlier_clusters_df['Monetoryvalue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Frequency'], palette=
sns.violinplot(y=outlier_clusters_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Recency'], palette=cl
```

```
sns.violinplot(y=outlier_clusters_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```







📊 Outlier Customer Segmentation

⭐ Cluster -1: **PAMPER** (Monetary Outliers)

Characteristics:
High spenders but not necessarily frequent buyers. Their purchases are large but infrequent.

Potential Strategy:
Focus on maintaining their loyalty with personalized offers or luxury services that cater to their high spending capacity.

🔁 Cluster -2: **UPSELL** (Frequency Outliers)

Characteristics:
Frequent buyers who spend less per purchase. These customers are consistently engaged but may benefit from upselling opportunities.

Potential Strategy:
Implement loyalty programs or bundle deals to encourage higher spending per visit, given their frequent engagement.

👑 Cluster -3: **DELIGHT** (Monetary & Frequency Outliers)

Characteristics:
The most valuable outliers, with extreme spending and frequent purchases. They are likely top-tier customers requiring special attention.

Potential Strategy:
Develop VIP programs or exclusive offers to maintain their loyalty and encourage continued engagement.

In [80]:
```
cluster_labels = {
    0: "RETAIN",
    1: "RE-ENGAGE",
    2: "NURTURE",
    3: "REWARD",
    -1: "PAMPER",
    -2: "UPSELL",
    -3: "DELIGHT"
}
```

In [81]:
```
full_clustering_df = pd.concat([non_outliers_df, outlier_clusters_df])

full_clustering_df
```

Out[81]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency	Cluster
0	12,346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12,347.00	1,323.32	2	2010-12-07 14:57:00	2	0
2	12,348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12,349.00	2,221.14	2	2010-10-28 08:23:00	42	0
4	12,351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4235	18,223.00	7,516.31	12	2010-11-17 12:20:00	22	-3
4236	18,225.00	7,545.14	15	2010-12-09 15:46:00	0	-3
4237	18,226.00	6,650.83	15	2010-11-26 15:51:00	13	-3
4241	18,231.00	4,791.80	23	2010-10-29 14:17:00	41	-3
4262	18,260.00	7,318.91	17	2010-11-30 12:25:00	9	-3

4285 rows × 6 columns

In [82]:

```
full_clustering_df["ClusterLabel"] = full_clustering_df["Cluster"].map(cluster_labels)

full_clustering_df
```

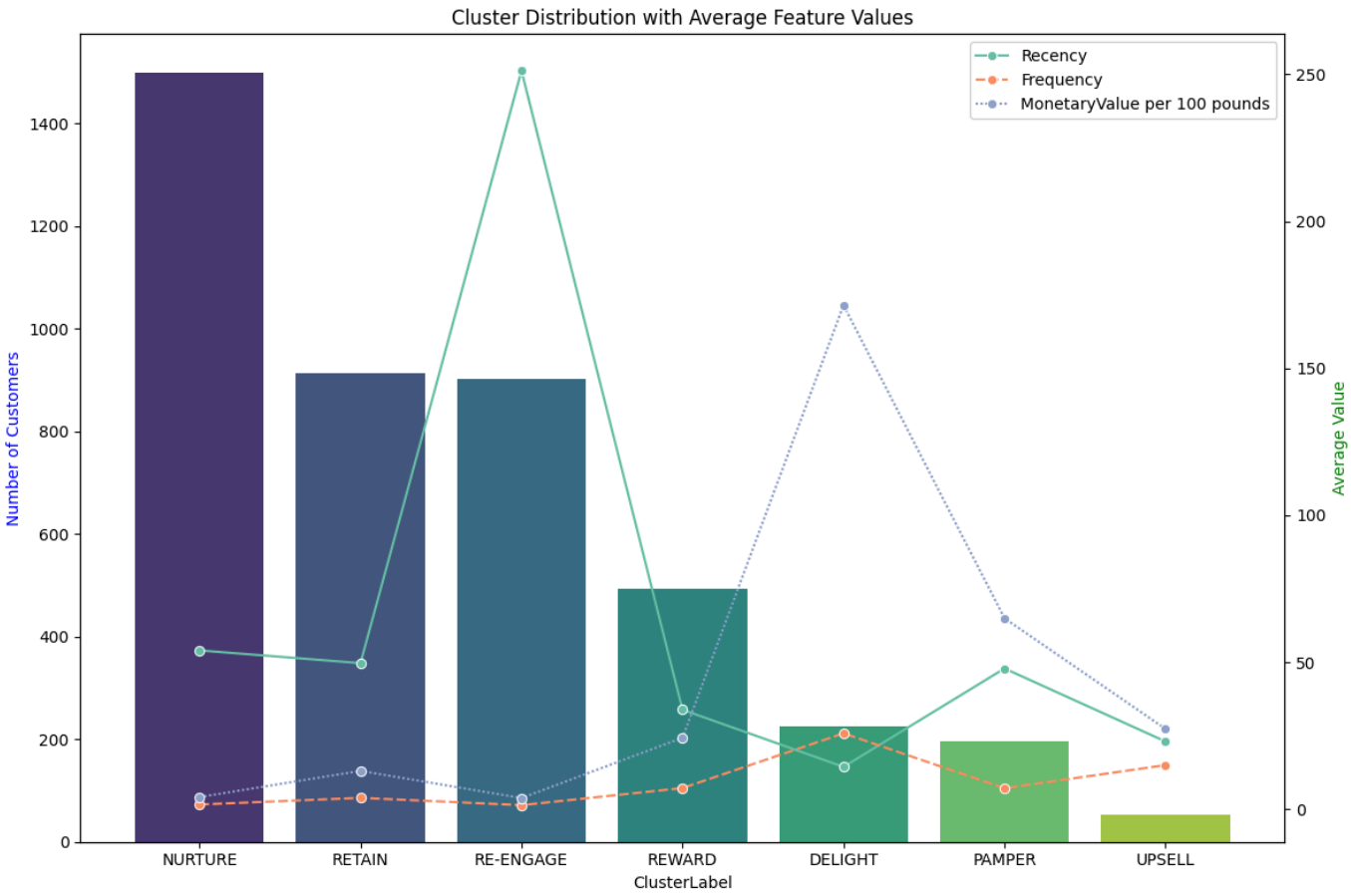
Out[82]:

	Customer ID	Monetoryvalue	Frequency	LastInvoicedate	Recency	Cluster	ClusterLabel
0	12,346.00	169.36	2	2010-06-28 13:53:00	164	1	RE-ENGAGE
1	12,347.00	1,323.32	2	2010-12-07 14:57:00	2	0	RETAIN
2	12,348.00	221.16	1	2010-09-27 14:59:00	73	2	NURTURE
3	12,349.00	2,221.14	2	2010-10-28 08:23:00	42	0	RETAIN
4	12,351.00	300.93	1	2010-11-29 15:23:00	10	2	NURTURE
...	...	...	...	...	...	...	...
4235	18,223.00	7,516.31	12	2010-11-17 12:20:00	22	-3	DELIGHT
4236	18,225.00	7,545.14	15	2010-12-09 15:46:00	0	-3	DELIGHT
4237	18,226.00	6,650.83	15	2010-11-26 15:51:00	13	-3	DELIGHT
4241	18,231.00	4,791.80	23	2010-10-29 14:17:00	41	-3	DELIGHT
4262	18,260.00	7,318.91	17	2010-11-30 12:25:00	9	-3	DELIGHT

4285 rows × 7 columns

Visualisation

In [85]:
```
cluster_counts = full_clustering_df['ClusterLabel'].value_counts()
full_clustering_df["MonetaryValue per 100 pounds"] = full_clustering_df["Monetoryvalue"] / 10
feature_means = full_clustering_df.groupby('ClusterLabel')[['Recency', 'Frequency', 'Monetary

fig, ax1 = plt.subplots(figsize=(12, 8))

sns.barplot(x=cluster_counts.index, y=cluster_counts.values, ax=ax1, palette='viridis', hue=c
ax1.set_ylabel('Number of Customers', color='b')
ax1.set_title('Cluster Distribution with Average Feature Values')

ax2 = ax1.twinx()

sns.lineplot(data=feature_means, ax=ax2, palette='Set2', marker='o')
ax2.set_ylabel('Average Value', color='g')

plt.tight_layout()
plt.show()
```



In [90]:
```
!jupyter nbconvert --to html /content/drive/MyDrive/Learning/DA-AB Testing/customer_segmentat
```